* Access Modifier

1) Public → Access from anywhere
2) Private → Access In Same Class only
3) Protected
4) default (nothing put)] → Access In Same Package only

---

* Constructor [Entry Point]

1) Default Constructor → Does not take any Parameters and also it can be created automatically if there is not Parametrized Cons.

2) Parametrized Constructor → it takes Parameters

#Note You cannot call automatticaly created default Cons if there is Parametrized Const. As automatticaly default Cons won't be Created In this Case

---

* Encapsulation → making attributes Private and make Public method to control This attributes like setters and getters.

EX:
```
Public class Teacher {
    Private String name

    Public void setname(String name) {this.name=name;}
    Public int getName() { return name ; }
```

* Inheritance → (Is a) Relationship and it uses word extend

#note Java doesn't support multi-inheritance

Ex: Public class Student extends Persons: this means that class student has all att and methods in class Person (Student is a Person)

---

* Inhentance lifecycle of object

Ex: Student extends Person
Student S1 = new Student()

→ Creats object first from Person then object from student:
1st → default constructor of Person
2nd → default constructor of student

* Key word "Super" → to enter Parent class from Parametrized Constructor

#note If there isn't default const. in Parent class you should use word Super to enter through Parametrized const.
if you didn't use Super in this case it will compile error

Scanned with CamScanner

* Protected and default in Inheritance

Protected → You can access this attribute if you extend Parent class from any Package

default → must extend Parent class in same Package to access this att.

Ex:

Pack1

```
Class A { Protected int x = 10; }
Class B extends A { Sout (x); }
```

allowed ✓

Pack1 `Class A { Protected int x = 10; }`

Pack 2 `Class B extends A { Sout (x); }` allowed ✓

while

Pack1

```
Class A { int x = 10; }
Class B extends A { Sout (x); }
```

allowed ✓

Pack1 `Class A { int x = 10; }`

Pack 2 `Class B extends A { Sout (x); }`

Not allowed ✗✗

Class

- default → Can create object, doesn't have abstract methods, Can inherite from
- abstract → Cannot create object, Can have abstract method, Can inherite from
- final → Can create object, Cannot have abstract methods, Cannot inherite from

---

methods

- default
- abstract → Must override, and doesn't have implementation
- final → Cannot override

---

keyword "static"

- static att. → Its value is set for class not objects if any object changes its value the new value will appear to all objects from this class
- static method → Must deal with static att.

ClassName.att ;
ClassName.Method() ;

→ Ways of calling

---

* Final att. → must have initial value

* Interface → like Abstract class but all methods are abstract

Default of att in interface → static final Public
Default of methods in interface → Public abstract

Interface methods

1) default void FuncName();
   → Can Give method initial implementation and any class Implements this interface can override it

2) static void FuncName();
   → Can Give method initial implementation but any class implements this interface cannot override this method

---

* Class can Implement more than one Interface
* Interface can extend Interface or multiple Interfaces

Scanned with CamScanner

* Polymorphism

EX1:

```
Public class Animal {
    public void print() { Sout ("Hello Animal"); }
}

Public class Cat extends Animal {
@override
Public void Print() { Sout ("Hello Cat"); }
}

Public class Dog extends Animal {
@override
Public void Print() { Sout ("Hello Dog"); }
```

EX2: From Same classes In EX1

```
Animal animal = new Cat();
    animal.Print() → Print in Cat
animal = new Dog();
    animal.print() → Print in Dog
```

Scanned with CamScanner